

DATABASE MANAGEMENT SYSTEMS

B.Tech IV Semester



Prepared
By
Dr V.SITHARAMULU

GITAM (Deemed to be University)

GITAM, Rudraram, Patancheru mandal, Hyderabad - 502329 Telangana, India

MODULE-I : INTRODUCTION TO DBMS AND DATABASE DESIGN

Introduction to DBMS: File system vs DBMS, advantages of DBMS, storage data, queries, DBMS structure, Types of Databases– Hierarchical, Network, Relational, Key-Value, Object-oriented, XML DB.

Overview of File Structures in database.

E-R model: Entities, attributes and entity sets, relationship and relationship set, mapping cardinalities, keys, features of ER model, conceptual database design with ER model.

MODULE-II : RELATIONAL MODEL AND BASIC SQL

Relational model: Integrity constraints over relations and enforcement, querying relational data, logical database design, views, destroying/altering tables and views.

Basic SQL: Introduction to SQL, Basic SQL Queries: DML, DDL, DCL, TCL

MODULE-III ADVANCES SQL AND PL/SQL

Structured Query Language (SQL): Select Commands, Union, Intersection, Except, Nested Queries, Aggregate Operators, Null values, Relational set operators, SQL join operators

Relational Algebra(RA): Selection, Projection, Set operations, Joins

Relational Calculus (TRC, DRC): Tuple Relational Calculus, Domain Relational Calculus

PL/SQL, Assertions, Triggers

MODULE-IV : SCHEMA REFINEMENT AND NORMAL FORMS

Introduction to Schema Refinement, Functional Dependencies, Reasoning about Functional Dependencies. Normal Forms, Properties of Decomposition, Normalization, different types of dependencies.

MODULE-V INTRODUCTION TO TRANSACTION MANAGEMENT, CONCURRENCY CONTROL AND CRASH RECOVERY

Introduction to Transaction Management: ACID properties, Transactions and Schedules, Concurrent Execution of Transactions, Lock-Based Concurrency Control.

Concurrency Control: 2PL, Serializability and Recoverability, Introduction to Lock Management, Lock Conversions, Dealing with Deadlocks, Concurrency control without locking.

Crash Recovery: Aries, Recovering from a System Crash

TEXT BOOKS:

1. Raghu Ramakrishnan and Johannes Gehrke, Database Management Systems, 3rd Edition, McGraw-Hill, 2014, 978-8131769591.
2. H.F.Korth and A.silberschatz, Database System Concepts, 7th Edition, McGraw-Hill, 2021, 978-9390727506.

REFERENCE BOOKS:

1. D. Ullman,, Principles of Database and Knowledge – Base Systems, Vol 1, 1st Edition, Computer Science Press, 1990
2. Ramez Elmasri, Shamkant B. Navathe, Fundamentals of Database Systems, Pearson Education, 2016

Schema refinement is a crucial concept in **database design**, especially in the context of relational databases. It focuses on improving a database schema to ensure it is **efficient, consistent, and free from anomalies**.

What is Schema Refinement?

Schema refinement is the process of **analyzing and improving a relational schema** by restructuring it into better forms (called **normal forms**) to:

- Reduce **data redundancy**

- Eliminate **update anomalies**

- Ensure **data integrity**

Why is Schema Refinement Needed?

When a database is poorly designed, it may lead to:

Redundancy – Repeated storage of the same data

Insertion Anomaly – Cannot insert data without other unrelated data

Deletion Anomaly – Deleting one record removes useful data

Update Anomaly – Updating data in multiple places leads to inconsistency

Schema refinement solves these issues by organizing data properly.

Functional Dependencies (FDs)

A functional dependency defines a relationship between attributes.

Example:

If **EmpID** → **EmpName**, then EmpID uniquely determines EmpName.

Schema refinement

Key Details of Functional Dependency

- **Definition:** If a student ID (Stu_Id) uniquely identifies a student's name (Stu_Name), then Stu_Id functionally determines Stu_Name ($Stu_Id \rightarrow Stu_Name$).
- **Determinant vs. Dependent:** In $X \rightarrow Y$, X is the determinant (the column holding the identifier) and Y is the dependent (the column holding the value, often a non-key column).
- **Types:**
 - **Trivial:** $A \rightarrow B$ is trivial if B is a subset of A (e.g., $\{Name, Age\} \rightarrow Name$).
 - **Non-Trivial:** $A \rightarrow B$ is non-trivial if B is not a subset of A (e.g., $Id \rightarrow Name$).
 - **Fully Functional:** $X \rightarrow Y$ is fully functional if Y cannot be determined by any proper subset of X .
 - **Transitive:** $A \rightarrow B$ and $B \rightarrow C$ determines $A \rightarrow C$.
- **Purpose:** FDs are vital for database normalization ($1NF, 2NF, 3NF, BCNF$) to reduce data redundancy, ensure data integrity, and identify candidate keys.

Trivial Functional Dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant. i.e., If $X \rightarrow Y$ and Y is the subset of X , then it is called a trivial functional dependency.

Symbolically: $A \rightarrow B$ is a trivial functional dependency if B is a subset of A .

The following dependencies are also trivial: $A \rightarrow A$ & $B \rightarrow B$

Schema refinement

Example 1 :

- $ABC \rightarrow AB$
- $ABC \rightarrow A$
- $ABC \rightarrow ABC$

Example 2:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent name is a subset of determinant set $\{\text{roll_no}, \text{name}\}$. Similarly, $\text{roll_no} \rightarrow \text{roll_no}$ is also an example of trivial functional dependency.

Schema refinement

In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is not a subset of X , then it is called Non-trivial functional dependency.

In Non-trivial functional dependency, the dependent is strictly not a subset of the determinant. i.e. If $X \rightarrow Y$ and Y is not a subset of X , then it is called Non-trivial functional dependency.

Example 1 :

- Id $\rightarrow$ Name
- Name $\rightarrow$ DOB

Example 2:

roll_no	name	age
42	abc	17
43	pqr	18
44	xyz	18

Here, $\text{roll_no} \rightarrow \text{name}$ is a non-trivial functional dependency, since the dependent name is not a subset of determinant roll_no. Similarly, $\{\text{roll_no}, \text{name}\} \rightarrow \text{age}$ is also a non-trivial functional dependency, since age is not a subset of $\{\text{roll_no}, \text{name}\}$

Normal Forms

Normal Forms

Normalization of data It is a process of analyzing the given relation schemas based on their FDs and primary keys to achieve the desirable properties of

(1) minimizing redundancy and

(2) minimizing the insertion, deletion, and modification anomalies (Update Anomalies). Unsatisfactory relation schemas that do not meet certain conditions (the normal form tests) are decomposed (break the relation into relation) into smaller relation schemas that meet the tests.

Normal Forms

Normalization is the process of minimizing **redundancy** from a relation or set of relations. Redundancy in relation may cause insertion, deletion, and update anomalies. So, it helps to minimize the redundancy in relations. **Normal forms** are used to eliminate or reduce redundancy in database tables.

1. First Normal Form –

If a relation contain composite or multi-valued attribute, it violates first normal form or a relation is in first normal form if it does not contain any composite or multi-valued attribute. A relation is in first normal form if every attribute in that relation is **singled valued attribute**.

- **Example 1** – Relation STUDENT in table 1 is not in 1NF because of multi-valued attribute STUD_PHONE. Its decomposition into 1NF has been shown in table 2.

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721, 9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 1



Conversion to first normal form

STUD_NO	STUD_NAME	STUD_PHONE	STUD_STATE	STUD_COUNTRY
1	RAM	9716271721	HARYANA	INDIA
1	RAM	9871717178	HARYANA	INDIA
2	RAM	9898297281	PUNJAB	INDIA
3	SURESH		PUNJAB	INDIA

Table 2

Normal Forms

- **Example 2 –**

ID	Name	Courses

1	A	c1, c2
2	E	c3
3	M	C2, c3

In the above table Course is a multi-valued attribute so it is not in 1NF.

Below Table is in 1NF as there is no multi-valued attribute

ID	Name	Course

1	A	c1
1	A	c2
2	E	c3
3	M	c2
3	M	c3

Normal Forms

Example: We have a relation with attributes, DEPARTMENT(Dname, Dnumber, Dmgr_ssn, Dlocation) with f.d
Dnumber → Dname, Dmgr_ssn, Dlocation

Let's populate some data in relation, the relation state will be:

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	Dlocations
Research	5	333445555	{Bellaire, Sugarland, Houston}
Administration	4	987654321	{Stafford}
Headquarters	1	888665555	{Houston}

From relation state it is clearly visible that the relation (DEPARTMENT) is not in 1NF because Dlocation is not atomic attribute. (one cell must contain only one value)

DEPARTMENT

Dname	<u>Dnumber</u>	Dmgr_ssn	<u>Dlocation</u>
Research	5	333445555	Bellaire
Research	5	333445555	Sugarland
Research	5	333445555	Houston
Administration	4	987654321	Stafford
Headquarters	1	888665555	Houston

2NF Normal Forms

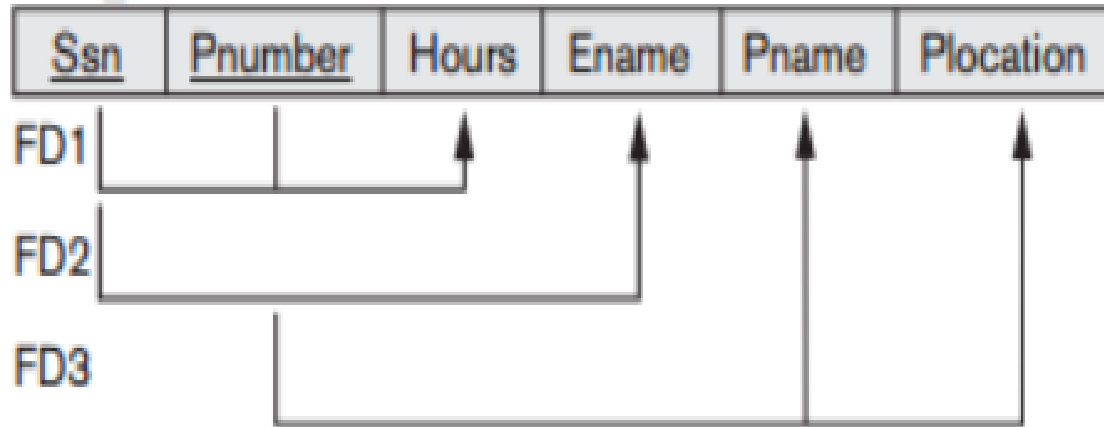
Second Normal Form (2NF)

- A relation schema R is in 2NF if it is in 1NF and if every nonprime attribute A in R is fully functionally dependent on the primary key of R.
- It is based on the concept of full functional dependency.
- A functional dependency $X \rightarrow Y$ is a full functional dependency if removal of any attribute A from X means that the dependency does not hold any more;

Normal Forms

Example:

EMP_PROJ



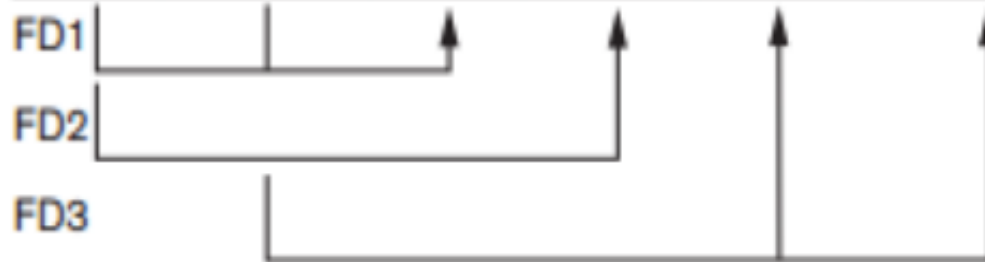
{Ssn, Pnumber} → Hours is a full dependency,
because neither Ssn → Hours nor Pnumber → Hours holds

Second Normal Form (2NF)

Normalizing EMP_PROJ relation into three relation EP1, EP2, EP3

EMP_PROJ

<u>Ssn</u>	<u>Pnumber</u>	Hours	Ename	Pname	Plocation
------------	----------------	-------	-------	-------	-----------



2NF Normalization

EP1

<u>Ssn</u>	<u>Pnumber</u>	Hours
------------	----------------	-------



EP2

<u>Ssn</u>	Ename
------------	-------



EP3

<u>Pnumber</u>	Pname	Plocation
----------------	-------	-----------



Second Normal Form (2NF)

Below is a **Course Enrollment** table of Datacamp students.

Student ID	Course ID	Course Name	Instructor Name
1001	201	SQL Fundamentals	Ken Smith
1002	202	Introduction to Python	Merlin O'Donnell
1001	202	Introduction to Python	Merlin O'Donnell

Here, the primary key is the composite of **Student ID** and **Course ID**. However, the non-prime attributes **Course Name** and **Course Fee** depend only on **Course ID**, not the entire key. This violates 2NF.

Steps for Decomposing Tables to Achieve 2NF

To make sure that a table follows the rules of 2NF, you need to:

1. **Identify All Candidate Keys:** Determine the minimal sets of attributes that uniquely identify rows in the table. These are your candidate keys.
2. **Determine Functional Dependencies:** Identify all functional dependencies in the table. Specifically, look for dependencies where non-prime attributes (those not part of any candidate key) depend only on a part of a composite key.
3. **Eliminate Partial Dependencies:** For each partial dependency:
 - Move the dependent attributes into a new table along with the part of the key they depend on.
 - Ensure the new table has a unique primary key.
4. **Repeat Until No Partial Dependencies Remain:** Confirm that every non-prime attribute in all tables is fully dependent on its respective primary key.

Second Normal Form (2NF)

Course enrollment table

Student ID	Course ID
1001	201
1002	202
1001	202

Course details table

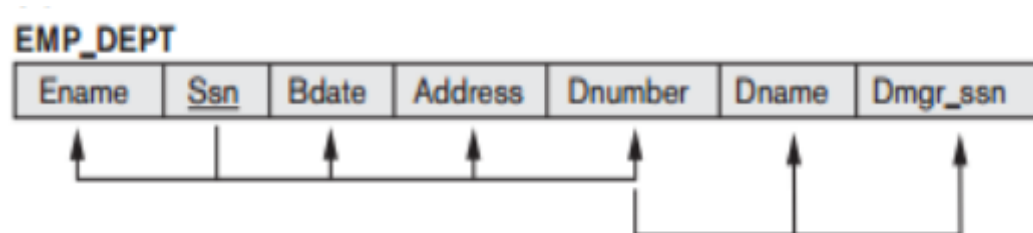
Course ID	Course Name	Instructor Name
201	SQL Fundamentals	Ken Smith
202	Introduction to Python	Merlin O'Donnell

Third Normal Form (3NF)

Third normal form (3NF) is based on the concept of **transitive dependency**.

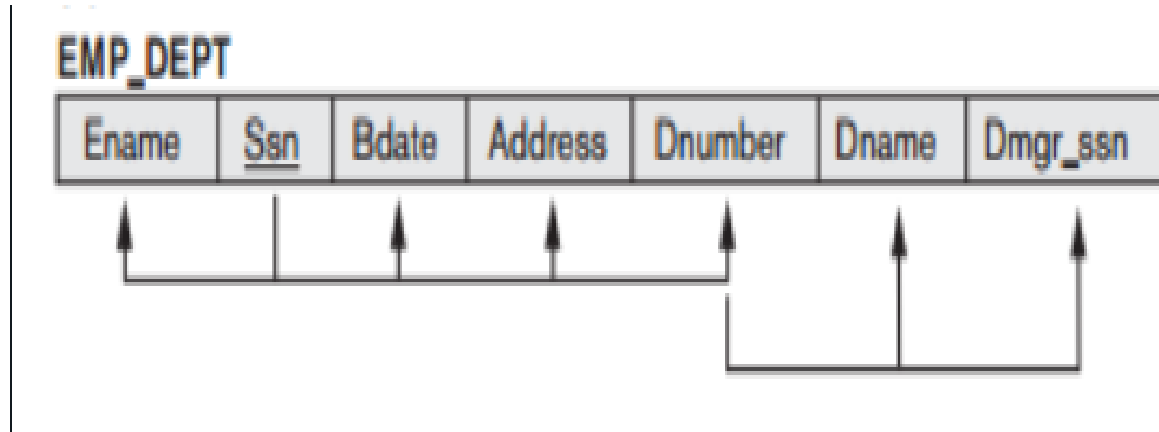
a relation schema R is in 3NF if it satisfies 2NF and no nonprime attribute of R is transitively dependent on the primary key.

- A functional dependency $X \rightarrow Y$ in a relation schema R is a transitive dependency if there exists a set of attributes Z in R that is neither a candidate key nor a subset of any key of R and both $X \rightarrow Z$ and $Z \rightarrow Y$ hold.
- Thus, a relation schema R is in 3NF if it satisfies 2NF and no nonprime attribute of R is transitively dependent on the primary key.

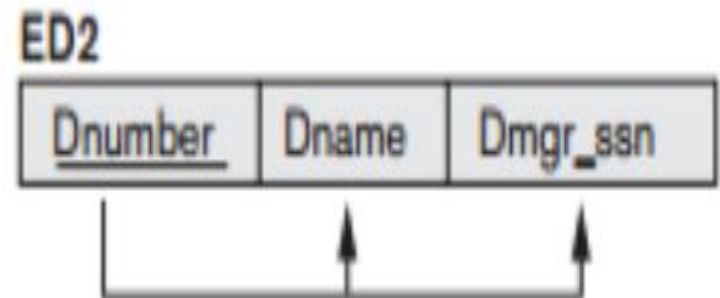
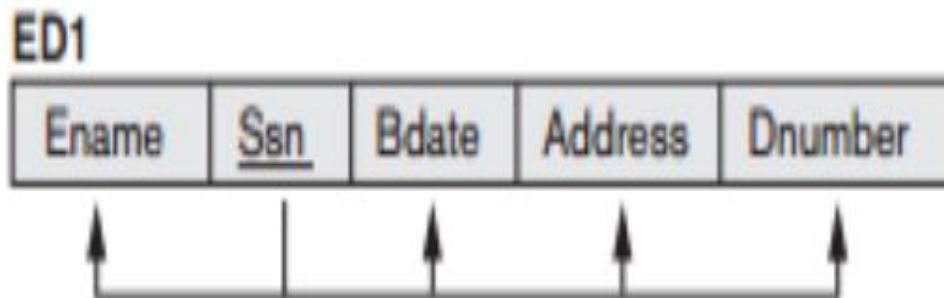


Third Normal Form (3NF)

Decomposing EMP_DEPT into ED1 and ED2



3NF Normalization



Boyce Codd Normal Form (BCNF)



Trivial Functional Dependency

In Trivial Functional Dependency, a dependent is always a subset of the determinant.

i.e. If $X \rightarrow Y$ and Y is the subset of X , then it is called trivial functional dependency

$\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$ is a trivial functional dependency, since the dependent name is a subset of determinant set $\{\text{roll_no}, \text{name}\}$

Non-trivial Functional Dependency

In **Non-trivial functional dependency**, the dependent is strictly not a subset of the determinant.

i.e. If $X \rightarrow Y$ and **Y is not a subset of X** , then it is called Non-trivial functional dependency.

Boyce Codd Normal Form (BCNF)

A relation schema R is in BCNF if whenever a nontrivial functional dependency $X \rightarrow Y$ holds in R, then X is a superkey of R.

Consider the below table

Student ID	Subject	Professor
1DT15ENG01	SQL	Prof. Mishra
1DT15ENG02	JAVA	Prof. Anand
1DT15ENG02	C++	Prof. Kanthi
1DT15ENG03	JAVA	Prof. Anand
1DT15ENG04	DBMS	Prof. Lokesh

- One student can enrol for multiple subjects.
- There can be multiple professors teaching one subject
- And, For each subject, a professor is assigned to the student

Boyce Codd Normal Form (BCNF)

As you can see **Student ID**, and **Subject** form the primary key, which means the **Subject** column is a **prime attribute**.

But, there is one more dependency, **Professor** → **Subject**. And while **Subject** is a prime attribute,

Professor is a **non-prime attribute**, which is not allowed by BCNF.

Now in order to satisfy the BCNF, we will be dividing the table into two parts.

One table will hold **Student ID** which already exists and newly created column **Professor ID**.

Boyce Codd Normal Form (BCNF)

Student ID	Professor ID
1DT15ENG01	1DTPF01
1DT15ENG02	1DTPF02
1DT15ENG02	1DTPF03
⋮	⋮

And in the second table, we will have the columns **Professor ID**, **Professor** and **Subject**.

Professor ID	Professor	Subject
1DTPF01	Prof. Mishra	SQL
1DTPF02	Prof. Anand	JAVA
1DTPF03	Prof. Kanthi	C++
⋮	⋮	⋮

Boyce Codd Normal Form (BCNF)

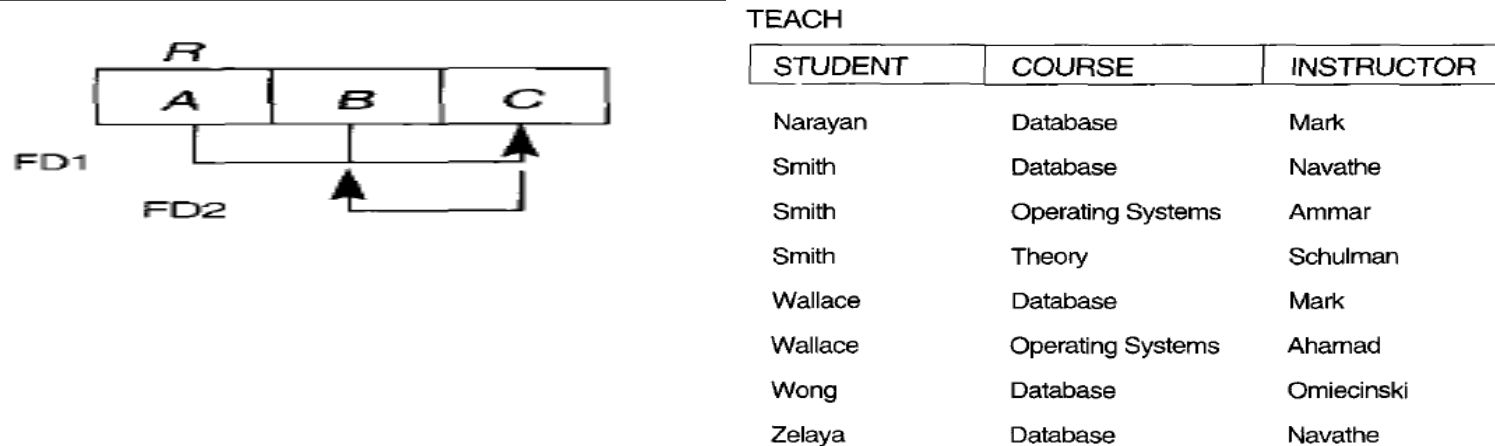


FIGURE 10.13 A relation TEACH that is in 3NF but not BCNF.

STUDENT as A, COURSE as B, and INSTRUCTOR as C. Hence this relation is in 3NF but not BCNF.

Decomposition of this relation schema into two schemas is not straight forward because it may be decomposed into one of the three following possible pairs:

1. {STUDENT, INSTRUCTOR} and {STUDENT, COURSE}.
2. {COURSE, INSTRUCTOR} and {COURSE, STUDENT}.
3. {INSTRUCTOR, COURSE} and {INSTRUCTOR, STUDENT}.

Fourth Normal Form normal form (4NF)

A relation R is in 4NF if and only if the following conditions are satisfied:

1. It should be in the Boyce-Codd Normal Form (BCNF).
2. the table should not have any Multi-valued Dependency.

For a dependency $A \twoheadrightarrow B$, if for a single value of A, multiple values of B exists, then the relation will be a multi-valued dependency.

(a) **EMP**

<u>ENAME</u>	<u>PNAME</u>	<u>DNAME</u>
Smith	X	John
Smith	Y	Anna
Smith	X	Anna
Smith	Y	John

(b) **EMP_PROJECTS**

<u>ENAME</u>	<u>PNAME</u>
Smith	X
Smith	Y

EMP_DEPENDENTS

<u>ENAME</u>	<u>DNAME</u>
Smith	John
Smith	Anna

The EMP relation with two MVDs $ENAME \twoheadrightarrow PNAME$ and $ENAME \twoheadrightarrow DNAME$.

Fourth Normal Form normal form (4NF)

Definition

A relation is in **4NF** if:

- It is already in **Boyce-Codd Normal Form (BCNF)**, and
- It has **no non-trivial multivalued dependencies**.

What is a Multivalued Dependency (MVD)?

An MVD occurs when:

One attribute determines **multiple independent values** of another attribute.

Notation:

$A \twoheadrightarrow B$

Means:

For one value of **A**, there can be **multiple values of B**, independent of other attributes.

Student	Subject	Hobby
John	Math	Cricket
John	Math	Music
John	Science	Cricket
John	Science	Music

Fourth Normal Form normal form (4NF)

Student	Subject	Hobby
John	Math	Cricket
John	Math	Music
John	Science	Cricket
John	Science	Music

Problem:

- A student can have multiple **Subjects**
- A student can have multiple **Hobbies**
- Subjects and hobbies are **independent**

MVDs:

Student $\twoheadrightarrow$ Subject

Student $\twoheadrightarrow$ Hobby

This creates redundancy (repeating combinations).

Fourth Normal Form normal form (4NF)

Convert to 4NF (Decomposition)

Student_Subject

Student	Subject
John	Math
John	Science

Student_Hobby

Student	Hobby
John	Cricket
John	Music

- No redundancy
- No multivalued dependency
- Relation is in **4NF**

Normal Forms

join dependency : is a type of database constraint that dictates a table can be accurately reconstructed by joining multiple smaller sub-tables (projections). It represents a lossless-join decomposition, ensuring no spurious data or data loss occurs during this process.

Lossless joining (or lossless decomposition) :in DBMS is a decomposition of a relation into smaller sub-relations that ensures no data is lost or created when the sub-relations are joined back together using a natural join, reconstructing the exact original table

Fifth Normal Form normal form (5NF)

- A relation is in 5NF if it is in 4NF and not contains any join dependency and joining should be lossless.
- 5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.
- 5NF is also known as Project-join normal form (PJ/NF).

SUPPLY

SNAME	PARTNAME	PROJNAME
Smith	Bolt	ProjX
Smith	Nut	ProjY
Adamsky	Bolt	ProjY
Walton	Nut	ProjZ
Adamsky	Nail	ProjX
Adamsky	Bolt	ProjX
Smith	Bolt	ProjY

R1

SNAME	PARTNAME
Smith	Bolt
Smith	Nut
Adamsky	Bolt
Walton	Nut
Adamsky	Nail

R2

SNAME	PROJNAME
Smith	ProjX
Smith	ProjY
Adamsky	ProjY
Walton	ProjZ
Adamsky	ProjX

R3

PARTNAME	PROJNAME
Bolt	ProjX
Nut	ProjY
Bolt	ProjY
Nut	ProjZ
Nail	ProjX

Normal Forms

Question 1

Consider the relation

$R(A, B, C, D, E)$

The set of functional dependencies is:

$A \rightarrow BC$

$CD \rightarrow E$

$B \rightarrow D$

$E \rightarrow A$

1(a) Find the closure of A^+ and CD^+ .

Normal Forms

1(a) Find the closure of A^+ and CD^+ .

Closure of A^+ :

Start with $A^+ = \{A\}$

$A \rightarrow BC \Rightarrow A^+ = \{A, B, C\}$

$B \rightarrow D \Rightarrow A^+ = \{A, B, C, D\}$

$CD \rightarrow E \Rightarrow A^+ = \{A, B, C, D, E\}$

So,

$A^+ = \{A, B, C, D, E\}$

Closure of CD^+ :

Start with $CD^+ = \{C, D\}$

$CD \rightarrow E \Rightarrow \{C, D, E\}$

$E \rightarrow A \Rightarrow \{C, D, E, A\}$

$A \rightarrow BC \Rightarrow \{C, D, E, A, B\}$

So,

$CD^+ = \{A, B, C, D, E\}$

Normal Forms

Question: Consider the relation

$R(A, B, C, D, E)$

The set of functional dependencies is:

$A \rightarrow BC$

$CD \rightarrow E$

$B \rightarrow D$

$E \rightarrow A$

1(b) Identify all the **candidate keys** of the relation.

Ans :

Candidate Key

A candidate key is an attribute (or set) whose closure gives all attributes.

$A^+ = \text{all attributes} \Rightarrow \mathbf{A \text{ is a candidate key}}$

$CD^+ = \text{all attributes} \Rightarrow \mathbf{CD \text{ is also a candidate key}}$

So,

Candidate Keys = $\{A\}, \{CD\}$

Normal Forms

Question 1:

Consider the relation

R(A, B, C, D, E)

The set of functional dependencies is:

$A \rightarrow BC$

$CD \rightarrow E$

$B \rightarrow D$

$E \rightarrow A$

1(c) Check whether the relation is in **2NF** and **3NF**. If not, normalize it.

ANS: **Check for 2NF:**

No partial dependency on part of a key (since A is single attribute key)

So, relation is in **2NF**

Normal Forms

Check for 3NF:

Check each FD:

$A \rightarrow BC$ (OK, A is key)

$CD \rightarrow E$ (CD is key, OK)

$B \rightarrow D$ (**Violation**, B is not a key)

$E \rightarrow A$ (**Violation**, E is not a key)

So, relation is **NOT in 3NF**

Decomposition to 3NF

Decompose based on violating dependencies:

R1(B, D) from $B \rightarrow D$

R2(E, A) from $E \rightarrow A$

R3(A, B, C, E) remaining attributes

Normal Forms

Question2: Consider the relation:

STUDENT (SID, SNAME, CID, CNAME, INSTRUCTOR)

Functional Dependencies:

$SID \rightarrow SNAME$

$CID \rightarrow CNAME, INSTRUCTOR$

$SID, CID \rightarrow \text{all attributes}$

2(a) Find (SID, CID)⁺

Start with:

$(SID, CID)^+ = \{SID, CID\}$

Apply FDs:

$SID \rightarrow SNAME \Rightarrow \{SID, CID, SNAME\}$

$CID \rightarrow CNAME, INSTRUCTOR \Rightarrow \{SID, CID, SNAME, CNAME, INSTRUCTOR\}$

Now we have all attributes.

So,

$(SID, CID)^+ = \{SID, CID, SNAME, CNAME, INSTRUCTOR\}$

Normal Forms

2 **question**: Consider the relation:

STUDENT (SID, SNAME, CID, CNAME, INSTRUCTOR)

Functional Dependencies:

$SID \rightarrow SNAME$

$CID \rightarrow CNAME, INSTRUCTOR$

$SID, CID \rightarrow \text{all attributes}$

2(b). Identify the **candidate key(s)**

Ans: Check closures:

SID⁺:

$SID \rightarrow SNAME \Rightarrow \{SID, SNAME\}$

(Not all attributes)

CID⁺:

$CID \rightarrow CNAME, INSTRUCTOR \Rightarrow \{CID, CNAME, INSTRUCTOR\}$

(Not all attributes)

(SID, CID)⁺:

Gives all attributes

So,

Candidate Key = {SID, CID}

2. Question Consider the relation:

STUDENT (SID, SNAME, CID, CNAME, INSTRUCTOR)

Functional Dependencies:

$SID \rightarrow SNAME$

$CID \rightarrow CNAME, INSTRUCTOR$

$SID, CID \rightarrow \text{all attributes}$

2(c) Check whether the relation is in **2NF**

Step 1: Prime attributes

SID, CID (part of candidate key)

Step 2: Non-prime attributes

SNAME, CNAME, INSTRUCTOR

Step 3: Check partial dependency

$SID \rightarrow SNAME$ (partial dependency)

$CID \rightarrow CNAME, INSTRUCTOR$ (partial dependency)

Since non-prime attributes depend on part of the key,

So,

Relation is NOT in 2NF

Normal Forms

2 question: Consider the relation:

STUDENT (SID, SNAME, CID, CNAME, INSTRUCTOR)

Functional Dependencies:

$SID \rightarrow SNAME$

$CID \rightarrow CNAME, INSTRUCTOR$

$SID, CID \rightarrow \text{all attributes}$

2(d) Normalize up to 3NF

Normalize up to 3NF

Step 1: Remove partial dependencies (convert to 2NF)

Decompose into:

R1(SID, SNAME)

$SID \rightarrow SNAME$

R2(CID, CNAME, INSTRUCTOR)

$CID \rightarrow CNAME, INSTRUCTOR$

R3(SID, CID)

Key relation

Normal Forms

Step 2: Check for 3NF

R1:

SID $\rightarrow$ SNAME (SID is key) $\Rightarrow$ OK

R2:

CID $\rightarrow$ CNAME, INSTRUCTOR (CID is key) $\Rightarrow$ OK

R3:

No non-trivial FD $\Rightarrow$ OK